Pembagian Tugas

ETS Praktik Programming Language Pragmatics: Ekstraktor CRUD Matrix

Kelompok	Ghaisan Khoirul Badruzaman dan Hafiz Fauzan Syafrudin
Kelas	2B D4 Teknik Informatika, Politeknik Negeri Bandung
Repository	https://github.com/Finerium/ekstraktor-crud-matrix (public)
Deadline	**Sabtu, 7 November 2026, pukul 17.00 WIB**, lewat link pengumpulan dari dosen
Versi dokumen	5 November 2026, ditulis setelah bagian Ghaisan selesai dan di-push

Inti pembagian (50/50)

- **Ghaisan** memegang **lexer dan parser** (Bab 4, 5, 6). Ini sama dengan jam ke-2 dan ke-3 di rencana 6 jam dari dosen. **Status: selesai.**
- **Hafiz** memegang **resolver, analyzer, dan data** (Bab 11). Ini sama dengan jam ke-4 dan ke-5 di rencana dosen. **Status: mulai sekarang**, dari commit terakhir di branch `main`.
- Jam ke-1 (membaca spesifikasi) dan jam ke-6 (laporan, latihan demo) dikerjakan berdua.
- Bagian Hafiz dikerjakan sesudah bagian Ghaisan karena resolver dan analyzer butuh lexer dan parser yang sudah benar. Dengan urutan ini AI agent Hafiz tidak menabrak pekerjaan Ghaisan.

1. Ringkasan tugas

Tool ini membaca file SQL lalu menghasilkan **CRUD matrix**, yaitu tabel mana yang dibuat (C), dibaca (R), diubah (U), dan dihapus (D) oleh setiap file. Alurnya mengikuti buku Crafting Interpreters: lexer (Bab 4), grammar dan AST (Bab 5), parser (Bab 6), dan resolver (Bab 11), lalu analyzer yang menerapkan aturan CRUD. Codebase dari dosen sengaja belum lengkap (3 TODO) dan berisi beberapa bug yang jumlahnya tidak diumumkan.

Aturan penting dari deskripsi tugas:

- `SPESIFIKASI.md` adalah acuan kebenaran. Kalau kode, komentar, dan spesifikasi berbeda, spesifikasi yang benar.
- Nilai bug dan TODO dari **test tersembunyi**. Semua test terlihat lulus belum berarti selesai.
- Tidak boleh mengubah API publik (SPESIFIKASI bagian 7), test bawaan di `tests/`, dan `baseline/regex_extractor.py`.
- `data/ground_truth.csv` untuk `audit.sql` dan `pemeliharaan.sql` dilabel **manual** dari spesifikasi, bukan menyalin output tool.
- AI boleh dipakai, tetapi **kedua anggota** harus bisa menjelaskan setiap perbaikan. Saat demo dosen memilih siapa yang menjawab.
- Setiap pemakaian AI dicatat di `docs/LOG_AI.md`. Bagian akar masalah di laporan bug wajib memakai konsep dari buku.
- Kode dan komentar dibuat sesederhana mungkin, mengikuti contoh komentar di deskripsi tugas.

Deliverable dan pemiliknya

No	Deliverable	Lokasi	Pemilik
1	Kode diperbaiki dan dilengkapi, ada commit dari kedua anggota	repository Git	Ghaisan: lexer, parser. Hafiz: resolver, analyzer
2	Satu entri laporan per bug	`docs/LAPORAN_BUG.md`	Bug A: Ghaisan. Bug B: Hafiz

No	Deliverable	Lokasi	Pemilik
3	Lima keputusan desain dan alternatif yang ditolak	`docs/KARTU_DESAIN.md`	Kartu 1 sampai 3: Ghaisan. Kartu 4 dan 5: Hafiz
4	Test regresi tiap bug dan 5 test kasus tepi	`tests/test_tambahan.py`	Regresi: pemilik bug. Kasus tepi: Ghaisan 2, Hafiz 3
5	Label `audit.sql` dan `pemeliharaan.sql`	`data/ground_truth.csv`	Hafiz
6	Hasil evaluasi, 3 kasus regex salah, 3 batas pendekatan	`docs/LAPORAN_AKHIR.md`	Bagian 1 sampai 3: Hafiz. Bagian 4: Ghaisan (sudah)
7	Log penggunaan AI	`docs/LOG_AI.md`	Masing-masing menulis barisnya sendiri
8	Demo 15 menit, tiap anggota diuji terpisah	sesi demo	Berdua

Bobot penilaian

Komponen	Bobot	Cara menilai
Bug diperbaiki	25%	Test tersembunyi
TODO diselesaikan	15%	Test tersembunyi
Laporan bug, kartu desain, test tambahan	20%	Ketepatan akar masalah dan kualitas test
Demo individual	35%	Prediksi output, modifikasi langsung, pertanyaan konsep
Log AI	5%	Kejujuran dan bukti verifikasi

2. Kondisi awal codebase

Di kode asli dari dosen (commit `279b7e9`), `python -m pytest` menghasilkan **25 lulus, 5 gagal**. Pembacaan `SPESIFIKASI.md` baris per baris terhadap seluruh kode menemukan 3 TODO dan 9 bug. Semua bug di bawah sudah **dibuktikan** dengan menjalankan potongan SQL kecil di kode asli, bukan dugaan. Bug lain masih mungkin ada (dosen sengaja tidak mengumumkan jumlahnya).

Kode	Tahap	Masalah singkat	Spesifikasi	Pemilik
A1	lexer	keyword huruf kecil dibaca IDENT	2.2	Ghaisan
A2	lexer	`<>` dipecah jadi dua token	2.5	Ghaisan
A3	lexer	`''` di dalam string memotong string	2.4	Ghaisan
T3	lexer	TODO komentar blok `/* ... */`	2.3	Ghaisan
A4	parser	`AND` dan `OR` setingkat	3, 3.1	Ghaisan
A5	parser	error recovery membuang token sampai akhir file	6	Ghaisan
A6	parser	`LEFT OUTER JOIN` ditolak	3	Ghaisan
T1	parser	TODO `INSERT INTO ... SELECT`	3, 5	Ghaisan
T2	resolver	TODO kolom tanpa qualifier	4.3	Hafiz
B1	resolver	qualifier dicari dari scope terluar dulu	4.3	Hafiz
B2	resolver	scope `DELETE` tidak ditutup	4.1, 4.5	Hafiz
B3	analyzer	subquery di `UPDATE` ditandai U, harusnya R	5	Hafiz

Lima test terlihat yang gagal di kode asli dan penyebabnya:

Test terlihat yang gagal	Penyebab	Status
`test_keyword_huruf_kecil`	A1	lulus sejak commit A1
`test_komentar_blok`	T3	lulus sejak commit T3
`test_insert_select`	T1	lulus sejak commit T1
`test_kolom_tanpa_qualifier_satu_tabel`	T2	masih gagal, bagian Hafiz
`test_matrix_laporan_penjualan`	A2, T1, B3 (satu gejala, tiga bug)	masih gagal karena B3, bagian Hafiz

3. Kenapa pembagiannya begini, dan kenapa 50/50

Dasarnya rencana 6 jam dari dosen. Pembagian ini mengikuti saran jadwal di deskripsi tugas, jadi beban per orang setara dengan dua jam inti masing-masing ditambah dua jam bersama:

Jam	Kegiatan menurut deskripsi tugas	Dikerjakan oleh
1	Baca SPESIFIKASI.md, jalankan test, CLI, dan `--ast`, catat gejala aneh	Berdua
2	Lexer: perbaiki yang salah, selesaikan TODO lexer, tulis test regresi	**Ghaisan**
3	Parser: selesaikan TODO, cocokkan fungsi dengan grammar, presedensi, error recovery	**Ghaisan**
4	Resolver: selesaikan TODO, periksa aturan scope bagian 4 baris demi baris	**Hafiz**
5	Analyzer dan data: aturan CRUD bagian 5, label ground truth, jalankan `evaluate.py`	**Hafiz**
6	Laporan, kartu desain, log AI, latihan demo saling menjelaskan	Berdua

Kenapa dipotong per tahap pipeline:

- Tiap orang memegang file sendiri, jadi AI agent masing-masing tidak saling menimpa kode.
- Urutannya sesuai ketergantungan: resolver dan analyzer bekerja di atas AST hasil parser. Kalau parser masih salah, hasil resolver dan analyzer ikut salah dan sulit dicek.
- Saat demo, tiap orang menguasai satu rangkaian bab secara utuh (Bab 4 sampai 6, atau Bab 11), lalu belajar bagian partner dari commit yang rapi.

Timbangan beban kerja:

Komponen	Ghaisan	Hafiz
Bug dan TODO	6 bug kecil (masing-masing 1 sampai 10 baris) dan 2 TODO	3 bug, TODO T2 (TODO paling besar: 3 kasus dan aturan scope), plus mencari bug tersembunyi lain di resolver dan analyzer
Test tambahan	6 test regresi, 3 test TODO, 2 kasus tepi	test regresi bug B, 3 kasus tepi
Data	tidak ada	label manual 2 file ground truth, menjalankan dan membaca `evaluate.py`
Laporan	LAPORAN_BUG A1 sampai A6, kartu 1 sampai 3, LAPORAN_AKHIR bagian 4, README, dokumen ini	LAPORAN_BUG bug B, kartu 4 dan 5, LAPORAN_AKHIR bagian 1 sampai 3
Demo	menguasai bagian sendiri dan bagian Hafiz	menguasai bagian sendiri dan bagian Ghaisan

Jumlah bug Ghaisan memang lebih banyak, tetapi perbaikannya pendek-pendek. Bagian Hafiz punya TODO terbesar, seluruh pekerjaan data dan evaluasi, serta laporan akhir yang paling panjang. Dua sisi itu dibuat seimbang supaya totalnya sekitar 50/50.

4. BAGIAN GHAISAN: lexer dan parser (Bab 4, 5, 6). STATUS: SELESAI

File yang dipegang: `crud/lexer.py` dan `crud/parser.py`. Satu bug atau TODO dibuat satu commit, dan setiap test regresi sudah dicek **gagal** di commit sebelum perbaikan lalu **lulus** sesudahnya. Penjelasan lengkap (gejala, reproduksi, akar masalah, alternatif) ada di `docs/LAPORAN_BUG.md`.

A1. Keyword huruf kecil dibaca sebagai IDENT	
Lokasi	`crud/lexer.py`, `Lexer._word`
Gejala	`select id from pelanggan` jadi tiga IDENT, parser melapor `diharapkan awal statement ..., ditemukan 'select'`. Seluruh `pemeliharaan.sql` huruf kecil.
Aturan	SPESIFIKASI 2.2: keyword tidak peka huruf, nilainya dinormalisasi ke huruf besar.
Perbaikan	lexeme diubah ke huruf besar sebelum dicari di `KEYWORDS`; nilai keyword disimpan huruf besar.
Konsep buku	Bab 4.7 Reserved Words and Identifiers, Bab 4.2.1 Token type.
Test, commit	`test_bug_a1_keyword_tidak_peka_huruf_besar_kecil`, commit `209f669`

A2. Operator <> dipecah jadi dua token	
Lokasi	`crud/lexer.py`, `Lexer._scan_token` cabang `<`
Gejala	`a <> 1` jadi `<` lalu `>`; parser error `diharapkan ekspresi, ditemukan '>'`. SELECT baris 3 `laporan_penjualan.sql` hilang.
Aturan	SPESIFIKASI 2.5: maximal munch, `<>` satu token.
Perbaikan	setelah `<`, coba `=` lalu `>` dengan `_match`, baru jatuh ke `<`.
Konsep buku	Bab 4.5.2 Operators (lookahead dua karakter), maximal munch di Bab 4.7.
Test, commit	`test_bug_a2_operator_tidak_sama_dengan_satu_token`, commit `40ca17f`

A3. Dua kutip '' di dalam string memotong string	
Lokasi	`crud/lexer.py`, `Lexer._string`
Gejala	`'O''Brien'` jadi `STRING 'O'` dan `STRING 'Brien'`; parser error di `'Brien'`.
Aturan	SPESIFIKASI 2.4: `''` di dalam string mewakili satu kutip.
Perbaikan	saat ketemu `'`, intip karakter berikutnya; kalau `'` lagi, simpan satu kutip dan lanjut.
Konsep buku	Bab 4.6.1 String literals, Bab 4.2.2 Literal value (lexeme vs nilai), lookahead Bab 4.5.
Test, commit	`test_bug_a3_dua_kutip_dalam_string_jadi_satu_kutip`, commit `13fac6a`

T3. TODO komentar blok `/* ... */`	
Lokasi	`crud/lexer.py`, `Lexer._scan_token` cabang `/`
Aturan	SPESIFIKASI 2.3: lintas baris, tidak bersarang, boleh berisi `*`, tidak ditutup = `LexError`.
Implementasi	lewati `/*`, buang karakter sampai `*/` pertama; kalau file habis, `LexError` di posisi awal komentar.
Konsep buku	challenge 4 Bab 4 (versi tanpa nesting).
Test, commit	2 test `test_todo_t3_...` dan kasus tepi `test_tepi_komentar_blok_tidak_bersarang`, commit `5968b3f`

A4. `AND` dan `OR` punya presedensi yang sama	
Lokasi	`crud/parser.py`, `_logical` diganti `_or_expr` dan `_and_expr`
Gejala	`a = 1 OR b = 2 AND c = 3` jadi `(a = 1 OR b = 2) AND c = 3`. CRUD matrix tidak berubah, tetapi pohon salah (`audit.sql` baris 4 sampai 6, `pemeliharaan.sql` baris 6).
Aturan	SPESIFIKASI 3 dan 3.1: `or_expr` dan `and_expr` dua tingkat berbeda, AND lebih kuat.
Perbaikan	satu fungsi per tingkat presedensi, `_or_expr` memanggil `_and_expr`.
Konsep buku	Bab 6.1 Ambiguity and the Parsing Game (grammar bertingkat), Bab 6.2 Recursive Descent, Bab 9.3 Logical Operators (`logic_or`, `logic_and`).
Test, commit	`test_bug_a4_and_lebih_kuat_daripada_or`, commit `0c8c2d0`

A5. Error recovery membuang token sampai akhir file	
Lokasi	`crud/parser.py`, `Parser._synchronize`
Gejala	`campuran_error.sql`: setelah error baris 4, UPDATE baris 6 dan DELETE baris 8 ikut hilang.
Aturan	SPESIFIKASI 6: buang token sampai dan termasuk `;` berikutnya, tepat satu error per statement rusak.
Perbaikan	loop berhenti setelah token yang dibuang adalah `;`.
Konsep buku	Bab 6.3 Syntax Errors: panic mode (6.3.1) dan synchronizing (6.3.3).
Test, commit	`test_bug_a5_error_recovery_berhenti_di_titik_koma`, commit `658568e`

A6. `LEFT OUTER JOIN` ditolak	
Lokasi	`crud/parser.py`, `Parser._join`
Gejala	`... LEFT OUTER JOIN ...` error `diharapkan JOIN, ditemukan 'OUTER'`; SELECT di `pemeliharaan.sql` hilang.
Aturan	grammar: `join = [ "INNER" | "LEFT" [ "OUTER" ] ] "JOIN" table_ref "ON" expr`.
Perbaikan	setelah `LEFT`, `_match_kw("OUTER")`; `INNER OUTER` tetap ditolak.
Konsep buku	Bab 5.1.2 notasi bagian opsional, Bab 6.2 satu aturan satu fungsi.
Test, commit	`test_bug_a6_left_outer_join_diterima`, commit `b5f14f0`

T1. TODO `INSERT INTO ... SELECT`	
Lokasi	`crud/parser.py`, `Parser._insert`
Aturan	grammar `insert_stmt` bagian 3; AST `Insert(table, columns, values=[], query=Select)`.
Implementasi	memakai ulang `_select()`, hasilnya disimpan di `Insert.query`.
Test, commit	2 test `test_todo_t1_...` dan kasus tepi BETWEEN, commit `59cb894`

Hasil setelah bagian Ghaisan: `pytest` 40 lulus, 2 gagal (dua-duanya bagian Hafiz). `evaluate.py` recall parser naik dari 0.10 ke 0.90, precision 0.90. Laporan yang sudah ditulis: `LAPORAN_BUG.md` A1 sampai A6, `KARTU_DESAIN.md` 1 sampai 3, `LOG_AI.md` baris 1 sampai 6, `LAPORAN_AKHIR.md` bagian 4, dan README.

5. BAGIAN HAFIZ: resolver, analyzer, dan data (Bab 11). STATUS: MULAI SEKARANG

File yang dipegang: `crud/resolver.py`, `crud/analyzer.py`, `crud/pipeline.py`, `crud/cli.py`, dan `data/ground_truth.csv`. Item T2 dan B1 sampai B3 sudah dibuktikan bug-nya dengan reproduksi di bawah, jadi tinggal diperbaiki, dibuatkan test regresi, dan dijelaskan. Kerjakan berurutan dari atas.

T2. TODO kolom tanpa qualifier	
Lokasi	`crud/resolver.py`, akhir fungsi `Resolver._column` (bagian `TODO(T2)`)
Aturan	SPESIFIKASI 4.3: kolom tanpa qualifier hanya melihat scope **terdalam**. Tepat satu tabel: isi `col.resolved` dengan nama tabel lengkap (atau `<subquery alias>`). Lebih dari satu tabel: warning `kolom 'nama' ambigu tanpa skema`, `resolved` tetap `None`. Tidak ada tabel: warning.
Reproduksi	`SELECT nama FROM pelanggan WHERE aktif = 1`: sekarang `resolved` = `None`, harusnya `pelanggan`.
Hati-hati	`INSERT ... VALUES` tidak punya scope (`self.scope` bisa `None`), jadi masuk kasus "tidak ada tabel". Pakai fungsi `_target_of` yang sudah ada untuk nama tabel atau subquery.
Test	test terlihat `test_kolom_tanpa_qualifier_satu_tabel` harus lulus, ditambah test sendiri untuk kasus ambigu dan kasus tanpa tabel.
Konsep buku	Bab 11.3.3 Resolving variable expressions, Bab 11.1 Static Scope.

B1. Qualifier dicari dari scope terluar dulu (shadowing salah)	
Lokasi	`crud/resolver.py`, `Resolver._lookup`
Gejala	`SELECT x.id FROM pelanggan x WHERE x.id IN (SELECT x.pelanggan_id FROM pesanan x)`: kolom `x.pelanggan_id` di subquery ter-resolve ke `pelanggan`, harusnya `pesanan`. Contoh nyata `audit.sql` baris 12 sampai 14. CRUD matrix tidak berubah, tetapi `--kolom` salah.
Aturan	SPESIFIKASI 4.3: qualifier dicari dari scope **terdalam ke luar**, yang pertama ketemu dipakai (shadowing dan correlated subquery).
Petunjuk	lihat urutan loop di `_lookup` (ada `reversed`).
Konsep buku	Bab 11.1 Static Scope (shadowing), Bab 11.3.3 (`resolveLocal` berjalan dari scope terdalam).

B2. Scope `DELETE` tidak pernah ditutup	
Lokasi	`crud/resolver.py`, `Resolver._delete`
Gejala	`DELETE FROM sesi s WHERE s.id = 1; SELECT s.id FROM pengguna u`: statement kedua tidak menghasilkan diagnostik apa pun, harusnya error `alias 's' tidak terdefinisi`. Contoh nyata `audit.sql` baris 16 dan 18.
Aturan	SPESIFIKASI 4.5: setelah satu statement selesai di-resolve tidak boleh ada scope tersisa.
Petunjuk	bandingkan `_delete` dengan `_update`: setiap `_push` harus punya `_pop`.
Konsep buku	Bab 11.3.1 Resolving blocks (`beginScope` dan `endScope` selalu berpasangan).

B3. Subquery di `UPDATE` ditandai U, harusnya R	
Lokasi	`crud/analyzer.py`, `analyze_statement` cabang `ast.Update`
Gejala	`UPDATE pesanan SET x = 1 WHERE pid IN (SELECT id FROM pelanggan)` memberi `pelanggan: U`, harusnya `R`. Contoh nyata `laporan_penjualan.sql` baris 12 dan `audit.sql` baris 20. Ini yang membuat test terlihat `test_matrix_laporan_penjualan` masih gagal.
Aturan	SPESIFIKASI 5: `UPDATE` target U, tabel lain R (subquery di `SET` maupun `WHERE`). Membaca tabel lewat subquery **selalu** R.
Petunjuk	operasi yang dikirim ke `_collect` untuk nilai `SET` dan `WHERE`.

B4 dan seterusnya. Cari bug lain di file bagian Hafiz	
Cara	cocokkan `resolver.py` dengan SPESIFIKASI 4.1 sampai 4.6, `analyzer.py` dengan bagian 5, `cli.py` dengan format bagian 8, baris demi baris. Pakai `--ast` dan `--kolom` pada keempat file contoh.
Yang dicek	deklarasi alias dua kali (error), subquery di `FROM` tanpa alias (warning), resolusi ke `<subquery alias>`, subquery `FROM` tidak melihat tabel lain di `FROM` yang sama (4.4), urutan `CRUD` di keluaran, nama tabel huruf kecil dengan skema, format CSV.
Kalau ketemu	beri kode B4, B5, dan seterusnya, satu commit per bug, test regresi, entri LAPORAN_BUG.

Ground truth `audit.sql` dan `pemeliharaan.sql` (wajib manual)	
Lokasi	`data/ground_truth.csv`, format `file,baris,tabel,operasi`
Aturan	satu huruf operasi per baris (`RU` ditulis dua baris). `baris` = baris keyword awal statement. Nama tabel huruf kecil lengkap dengan skema (`crm.pelanggan`). Statement yang gagal di-parse tidak dihitung. Error resolver (misalnya alias tidak terdefinisi) tidak membatalkan statement.
Cara	buka file SQL, tentukan sendiri operasi tiap tabel per statement memakai tabel aturan SPESIFIKASI 5, baru setelah itu bandingkan dengan `python evaluate.py`. **AI tidak boleh mengisi file ini dari output tool.**

`docs/LAPORAN_AKHIR.md` bagian 1 sampai 3	
Bagian 1	tempel keluaran `python evaluate.py` setelah ground truth lengkap dan bug B sudah diperbaiki.
Bagian 2	tiga kasus regex salah (lihat daftar FP dan FN regex di keluaran `evaluate.py`), misalnya teks `FROM` di dalam komentar atau string, `DELETE FROM` yang juga terbaca R, statement rusak yang tetap dihitung. Kaitkan dengan konsep lexing, parsing, atau resolusi.
Bagian 3	tiga batas pendekatan yang tetap ada walau semua bug beres, misalnya tidak ada skema database (kolom tanpa qualifier di dua tabel tetap ambigu), view atau trigger yang menyembunyikan tabel asli, fitur di luar lingkup seperti CTE dan UNION. Sebutkan akibat ke CRUD matrix dan informasi yang dibutuhkan untuk mengatasinya.

Laporan lain dan test bagian Hafiz	
LAPORAN_BUG	entri B1, B2, B3 (dan B4+ kalau ada) di bawah judul BAGIAN HAFIZ, format sama dengan A1 sampai A6.
KARTU_DESAIN	kartu 4: urutan pencarian scope dan akibatnya pada shadowing dan correlated subquery. Kartu 5: kenapa kolom tanpa qualifier hanya dilihat di scope terdalam dan apa kelemahannya.
Test tambahan	di blok BAGIAN HAFIZ `tests/test_tambahan.py`: satu test regresi per bug B, dan 3 kasus tepi. Ide kasus tepi: correlated subquery yang merujuk alias luar, alias dideklarasikan dua kali, subquery `FROM` tanpa alias, tabel yang dibaca sekaligus diubah (urutan `RU`).
LOG_AI	baris mulai nomor 7, plus refleksi Hafiz.

6. Aturan kerja supaya tidak tabrakan

File	Pemilik	Catatan
`crud/lexer.py`, `crud/parser.py`	Ghaisan	sudah selesai; Hafiz jangan mengubah tanpa kabar
`crud/resolver.py`, `crud/analyzer.py`	Hafiz	
`crud/pipeline.py`, `crud/cli.py`	Hafiz	hanya kalau ada bug yang melanggar spesifikasi
`crud/ast_nodes.py`	tidak diubah	API publik (nama kelas dan field)
`data/ground_truth.csv`	Hafiz	manual, tanpa AI
`tests/test_tambahan.py`	berdua	masing-masing di bloknya sendiri
`docs/LAPORAN_BUG.md`, `docs/KARTU_DESAIN.md`, `docs/LOG_AI.md`	berdua	masing-masing di bagiannya
`docs/LAPORAN_AKHIR.md`	berdua	bagian 1 sampai 3 Hafiz, bagian 4 Ghaisan
`tests/test_*.py` bawaan, `baseline/`, `SPESIFIKASI.md`	**dilarang diubah**	saat penilaian `tests/` diganti salinan asli

1. Hafiz mulai dari commit terakhir di `main` (sudah berisi seluruh bagian Ghaisan). Selalu `git pull` sebelum mulai.
2. Satu bug atau TODO = satu commit. Pesan commit diawali kodenya, misalnya `TODO T2: ...` atau `Bug B1: ...`.
3. Test regresi ditulis lebih dulu dan dicek **gagal**, baru kode diperbaiki sampai test **lulus**.
4. Commit memakai akun GitHub Hafiz sendiri, karena deliverable meminta commit dari kedua anggota.
5. Kalau menemukan bug di file bagian Ghaisan, jangan langsung diubah. Kabari dulu, nanti diputuskan siapa yang memperbaiki.
6. Kode dan komentar sesederhana mungkin, gaya komentar seperti contoh di deskripsi tugas (komentar per langkah, bahasa Indonesia sederhana).
7. Jangan mengubah nama fungsi atau perilaku API publik (SPESIFIKASI bagian 7). Menambah fungsi, file, dan test boleh.
8. Sebelum push: `python -m pytest` harus lulus semua, termasuk test bagian Ghaisan.

7. Langkah git untuk Hafiz

```
# sekali saja: terima undangan collaborator di GitHub, lalu
git clone https://github.com/Finerium/ekstraktor-crud-matrix.git
cd ekstraktor-crud-matrix
git config user.name  "Hafiz Fauzan Syafrudin"
git config user.email "<email GitHub Hafiz>"
python -m pip install pytest
python -m pytest                 # kondisi awal: 40 passed, 2 failed

# setiap selesai satu bug atau TODO
python -m pytest
git add crud/resolver.py tests/test_tambahan.py docs/LAPORAN_BUG.md
git commit -m "Bug B1: qualifier dicari dari scope terdalam"
git pull --rebase
git push
```

8. Prompt siap pakai untuk AI agent Hafiz

Salin teks di bawah ke coding agent (Claude Code, Codex, Cursor, atau yang lain) setelah clone repo. Prompt ini membatasi agent ke file bagian Hafiz saja.

```
Kamu membantu Hafiz mengerjakan BAGIAN HAFIZ di repo ekstraktor-crud-matrix
(tugas ETS Programming Language Pragmatics). Bagian Ghaisan (lexer dan parser)
sudah selesai, jangan diubah.

Baca dulu: README.md, SPESIFIKASI.md (acuan kebenaran), docs/PEMBAGIAN_TUGAS.pdf
bagian 5 dan 6, dan isi docs/LAPORAN_BUG.md bagian Ghaisan sebagai contoh format.

Kerjakan berurutan, satu item satu commit:
1. TODO T2 di crud/resolver.py (Resolver._column), SPESIFIKASI 4.3.
2. Bug B1 (Resolver._lookup), B2 (Resolver._delete), B3 (analyzer.py, UPDATE).
3. Cari bug lain di crud/resolver.py, crud/analyzer.py, crud/pipeline.py,
   crud/cli.py dengan mencocokkan SPESIFIKASI bagian 4, 5, dan 8 baris demi baris.

Aturan:
- Untuk setiap bug: tulis test regresi dulu di tests/test_tambahan.py blok
  "BAGIAN HAFIZ", pastikan GAGAL, baru perbaiki sampai LULUS.
- Jangan ubah crud/lexer.py, crud/parser.py, crud/ast_nodes.py, tests/test_*.py
  bawaan, baseline/, SPESIFIKASI.md, dan API publik (SPESIFIKASI bagian 7).
- Kode dan komentar sesederhana mungkin, komentar per langkah dalam bahasa
  Indonesia sederhana, supaya mudah dijelaskan saat demo.
- JANGAN mengisi data/ground_truth.csv. Itu dilabel Hafiz secara manual.
- Tulis entri docs/LAPORAN_BUG.md (Bug B1 dst.) dengan akar masalah memakai
  konsep Crafting Interpreters Bab 11, dan catat semua pekerjaanmu di
  docs/LOG_AI.md mulai baris 7.
- Jalankan python -m pytest sebelum selesai. Semua test harus lulus.
```

9. Persiapan demo (35% nilai)

Demo 15 menit, **setiap anggota diuji terpisah**: prediksi output, modifikasi langsung, dan pertanyaan konsep. Dosen bebas menanyakan bagian mana saja ke siapa saja, jadi masing-masing wajib menguasai bagian partner. Cara belajar paling cepat: buka `git log`, lalu baca diff tiap commit partner satu per satu.

Daftar pertanyaan latihan

Pertanyaan	Bagian	Inti jawaban
Kenapa `select` huruf kecil harus jadi KEYWORD di lexer, bukan di parser?	A1	Lexer yang menentukan jenis token dari lexeme (Bab 4.7). Kalau salah di lexer, parser menerima token yang salah jenis.
Apa itu maximal munch? Contohnya di proyek ini?	A2	Ambil lexeme terpanjang yang cocok: `<>` dan `<=` satu token (Bab 4.5.2, 4.7).
Apa beda lexeme dan nilai literal pada `'O''Brien'`?	A3	Lexeme `'O''Brien'` (dengan kutip), nilai `O'Brien` (Bab 4.2.2).
Gambar pohon `a = 1 OR b = 2 AND c = 3`.	A4	Akar OR, kiri `a = 1`, kanan AND. Presedensi dari urutan fungsi (Bab 6.1).
Kenapa sinkronisasi di `;`, bukan di `SELECT`?	A5	`SELECT` bisa ada di dalam statement (subquery, INSERT SELECT), `;` tidak (Bab 6.3.3).
Fungsi parser mana yang berpadanan dengan aturan `join`? Apa yang dulu kurang?	A6	`_join`; bagian opsional `[ "OUTER" ]` belum diterjemahkan jadi `match`.
Kenapa komentar blok tidak bersarang? Apa hasil `/* a /* b */ c */`?	T3	Spesifikasi 2.3 dan 9. Hasilnya token `c`, `*`, `/`.
Kenapa kolom tanpa qualifier hanya dicari di scope terdalam?	T2	Aturan 4.3; tanpa skema database tidak tahu kolom milik tabel luar, jadi dibatasi supaya pasti.

Pertanyaan	Bagian	Inti jawaban
Apa itu shadowing? Tunjukkan di `audit.sql`.	B1	Alias `x` di subquery menutupi `x` di luar, cari dari scope terdalam (Bab 11.1).
Apa akibatnya kalau scope tidak di-pop?	B2	Alias statement sebelumnya terlihat di statement berikutnya, melanggar 4.5 (Bab 11.3.1).
Kenapa subquery di UPDATE ditandai R?	B3	Membaca tabel lewat subquery selalu R (SPESIFIKASI 5).

Latihan modifikasi langsung

- Tambah keyword baru (misalnya `HAVING`) ke `KEYWORDS` dan jelaskan efeknya ke lexer dan parser.
- Tambah `RIGHT JOIN` ke grammar: ubah `_join` dan kondisi `while` di `_select`.
- Tambah operator `||` di lexer, lalu tentukan tingkat presedensinya di parser.
- Prediksi keluaran `python -m crud.cli FILE --ast` untuk SQL baru yang ditulis partner, baru jalankan.

10. Checklist sebelum dikumpulkan (Sabtu, 7 November 2026, 17.00 WIB)

Cek	Pemilik	Status
Bug A1 sampai A6, TODO T1 dan T3, test, laporan bagian A	Ghaisan	selesai
TODO T2, bug B1 sampai B3, cari bug lain di resolver/analyzer	Hafiz	belum
`python -m pytest` lulus semua	berdua	belum (2 gagal, bagian Hafiz)
Test bawaan tidak berubah: `git diff 279b7e9 -- tests/test_lexer.py tests/test_parser.py tests/test_resolver.py tests/test_analyzer.py` kosong	berdua	sejauh ini aman
`data/ground_truth.csv` lengkap untuk 4 file, diisi manual	Hafiz	belum
`docs/LAPORAN_AKHIR.md` bagian 1 sampai 4	Hafiz, Ghaisan	bagian 4 selesai
`docs/KARTU_DESAIN.md` 5 kartu	Ghaisan 3, Hafiz 2	3 dari 5
`tests/test_tambahan.py`: regresi tiap bug dan 5 kasus tepi	Ghaisan 2, Hafiz 3	2 dari 5 kasus tepi
`docs/LOG_AI.md` lengkap dengan refleksi kedua anggota	berdua	bagian Ghaisan selesai
Ada commit dari akun Ghaisan dan akun Hafiz	berdua	baru Ghaisan
Link repository dikumpulkan lewat link pengumpulan dosen	Ghaisan	belum
Latihan demo saling menjelaskan bagian partner	berdua	belum